

FINDER: NONLINEAR OPTIMIZATION APPLICATION FOR SOLVING NATURAL LANGUAGE PROCESSING PROBLEMS

Paola Rodríguez
Laura Pamplona
David Sánchez

I. PROBLEM DESCRIPTION

Semantic similarity analysis between complex texts represents a significant challenge in natural language processing. Many texts contain deep relationships and meanings that cannot be captured by approaches limited to comparing words or surface patterns. This makes it difficult for systems to identify when two texts express similar ideas using different forms of wording.

II. INTRODUCTION

The rapid growth of textual information in digital environments has driven the development of automated methods that facilitate its analysis and understanding. Although multiple approaches exist within the field of natural language processing (NLP), limitations remain when accurately representing semantic similarity between different texts. In this context, the present project proposes an approach that projects texts into a vector space and constructs graphs that reflect their semantic relationships. This mathematical representation makes it possible to group texts with related meanings and visualize the connections between them, which is useful for classification and automated analysis tasks.

III. OBJECTIVES

III-A. General Objective

Use graph theory and vector representations of texts as tools to analyze and visualize semantic similarity between personal descriptions. By modeling these relationships as a graph, the aim is to identify patterns of conceptual proximity, understand how texts are grouped according to their meaning, and discover semantic connections that might not be apparent at first glance.

III-B. Specific Objectives

- Collect a set of personal descriptions to serve as a database for semantic similarity modeling and analysis.
- Implement a text preprocessing and vectorization process using embeddings, ensuring the preservation of semantic relationships between terms.
- Model the relationships between personal descriptions as a weighted graph, where nodes represent texts and edges indicate their degree of similarity.
- Develop visualizations that allow the graph results to be interpreted, identifying semantic clusters and non-obvious relationships between descriptions.

IV. THEORETICAL FRAMEWORK

IV-A. Background

Semantic similarity between texts has been explored extensively in the context of natural language processing. In its early stages, it was traditionally addressed through lexical or statistical approaches, such as term frequency or bag-of-words representations. These models treated texts as collections of words without context, and their main limitation was their inability to directly capture deep semantic relationships between words.

With the development of deep learning, defined by Jim Holdsworth (n.d.) as a "subset of machine learning that uses multilayer neural networks, called deep neural networks, to simulate the complex decision-making power of the human brain," embedding models were developed, such as:

- **Word2Vec:** It was a pioneering model that uses a neural network with three layers—an input layer, a hidden layer, and an output layer—which is trained on large amounts of text to adjust word vectors so that predictions are correct according to the given context.

This model is highly effective because it "effectively captures the semantic and arithmetic properties of a word" (DataScientist, 2022), and because it paved the way for many other models such as GloVe and BERT.

- **GloVe:** It is designed to capture semantic relationships between words by analyzing the frequency with which they occur in the same context.

It works by preprocessing the texts and dividing them into individual words. A vocabulary is then created based on the frequency of each word, followed by the construction of a co-occurrence matrix that counts how frequently a word appears near other words within a given context. If two words generally occur together or next to each other, their vectors will be close within the embedding space.

- **BERT:** Like Word2Vec and GloVe, it transforms words into vectors in a continuous space while preserving the semantic relationships between them. This model, however, uses bidirectional context to improve performance on natural language processing tasks.

These models represented a major advance because texts could no longer be compared only according to literal word matching, but also according to meaning.

A clear example of the use of embeddings is Tinder. Currently, this application uses artificial intelligence to process language through the Word2Vec model. By identifying similarity within language patterns, users can connect more easily with people who are similar to them based on behaviors and preferences expressed through language.

In the context of natural language processing, graphs have been implemented to model relationships between documents and different structures. Working with graphs in such contexts involves visualizing complex data structures, and using weighted graphs makes it visually easier to recognize connections between texts based on the weights assigned to edges according to their respective similarity.

Approaches such as the one presented in this project demonstrate that vector representations combined with graphs can capture conceptual proximity between texts.

IV-B. Theoretical Foundations

Graph theory is a branch of mathematics with applications in computer science that studies structures composed of vertices and edges connecting them. In the context of this project, graphs allow texts to be represented as nodes and semantic similarity relationships between them as weighted edges, facilitating the visualization and analysis of conceptual proximity between different personal descriptions.

- **Natural language processing:** Allows devices to recognize, generate, and understand text using machine learning, facilitating communication through human language.
- **Embeddings:** Vector representations of words or phrases in a continuous space, where semantic relationships between words can be represented.
- **Vector space:** Mathematical structure formed by vectors for which vector addition and scalar multiplication are defined. It allows texts to be represented as vectors in $\mathbb{R}^n$.
- **Euclidean distance:** Represents the shortest path between two points: $d(a, b) = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$. The smaller the distance, the greater the similarity between texts.
- **Semantic similarity:** Measures how similar two texts are in meaning beyond literal matches.
- **Weighted graphs:** Structures in which edges have an associated weight. In this project, that weight represents the degree of similarity between personal descriptions.

V. PROBLEM MODELING

The problem modeling proposed by FINDER stems from the need to structurally represent the semantic relationships existing between different texts or personal descriptions. In order to capture contextual similarities and quantify the degree of relationship between texts, a combination of *graph theory* and *vector representations* obtained through natural language processing (NLP) techniques is used.

V-A. Representation of the Text Set

Consider a set of text corpus representations denoted by T , where T is a subset of the universal set of texts TU . Each element of T corresponds to a description or document to be analyzed. From this set,

the **Graph** $G(T)$ is defined, which serves as a formal model of the semantic relationships between the texts.

The mathematical definition of the graph is given as follows:

$$\begin{aligned} \{n_i\}_{i=1}^{|T|} &= K \\ N(G(T)) &= K \\ E(G(T)) &= \{(a, b) \in K \times K \mid a \neq b\} \end{aligned}$$

Likewise, the following function is defined:

$$te = \{(n_i, t_i) \in K \times T\}$$

which assigns each graph node its corresponding text.

Graphical Example

The following is an example of graph $G(T)$ for a specific case, in which the set T contains three personal descriptions in different languages. Each node represents a text, and the function te associates each one with its corresponding description.

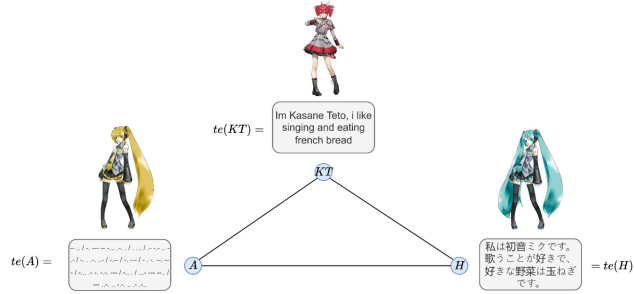


Figure 1: Representative graph of the modeling of three entities with their textual descriptions.

The graph shown represents the semantic relationship between three personal descriptions written in different languages. Each graph node corresponds to a specific text, and the edges reflect their degree of similarity, calculated using the Euclidean distance between their vector representations.

Proposed Algorithm

V-B. Vectorization Function

In order to quantify the context of each text, the following function is introduced:

$$Vec : T \rightarrow \mathbb{R}^n$$

which assigns each corpus a vector in an n -dimensional space. Each vector component represents semantic features derived from the text content,

obtained through *embedding* models or distributed representations.

This function allows us to compare texts written in different languages within the same vector space. Thus, texts with conceptually similar meanings will be located at short distances from one another, regardless of the language in which they are written.

For example, two texts about *cinema* will have a smaller Euclidean distance between their vectors than a text about *cinema* and another about *sports*.

V-C. Construction of the Semantic Graph

The proposed graph structure aims to represent the contextual similarities between the texts in set T through its edges. Each edge connects two nodes and is labeled with a value that quantifies the degree of similarity between the contexts of the texts it represents.

Formally, the weight of edge E_{ab} is defined using a similarity metric based on the Euclidean distance between the text vectors:

$$\begin{aligned} d(a, b) &= \sqrt{\sum_{i=1}^n (a_i - b_i)^2} \\ sim(a, b) &= \frac{1}{1 + d(a, b)} \end{aligned}$$

Thus, the smaller the values of $d(a, b)$, the greater the similarity between the associated texts, and vice versa. The set of weights $sim(E_{ab})$ provides a quantitative measure that makes it possible to interpret and visualize the semantic relationships between the texts.

The general procedure of the algorithm is as follows:

1. Iterate over the nodes n_i and vectorize the associated texts: $Vec(t_i)$.
2. For each pair of nodes (a, b) , calculate the Euclidean distance $dist_{euclid}(Vec(a), Vec(b))$.
3. Assign the calculated value to the edge with endpoints (a, b) .

The first step can be implemented using a depth-first or breadth-first traversal of the graph, depending on the chosen strategy.

[12pt]article [utf8]inputenc graphicx float amsmath amssymb caption

VI. PROPOSED SOLUTION

The proposed solution consists of developing, in Python, a system capable of evaluating semantic similarity between personal descriptions using natural language processing techniques and graph theory. To this end, the texts are preprocessed and transformed into vectors using an *embedding* model, allowing their semantic content to be represented quantitatively.

These vectors are used to construct a graph in which each node corresponds to a text and each edge indicates the degree of similarity between two descriptions. This structure is then used to analyze patterns, clusters, and relevant relationships within the dataset. Finally, a graph visualization is generated to facilitate interpretation of the semantic network, and conclusions are formulated based on the results obtained.

VII. SOLUTION IMPLEMENTATION

The FINDER system was implemented using the Python programming language, together with various libraries for natural language processing, semantic similarity analysis, and graph construction. The following sections provide a detailed description of each stage of the modeling process.

VII-A. Work Environment Setup

To ensure adequate performance, GPU availability was verified using the `torch`. This acceleration made it possible to process the embedding-generation model efficiently. The following libraries were also installed:

- **Unsloth**: used to load the Qwen2 7B embedding model in an optimized manner.
- **NetworkX**: used for constructing, manipulating, and analyzing the semantic graph.
- **Pandas**: used for reading and cleaning the CSV file.
- **PyForceAtlas2**: responsible for generating the graph's spatial layout.
- **Matplotlib**: used for the final visualization of the graph.

This set of tools provided a robust environment for corpus processing.

VII-B. Loading and Cleaning the Text Set

The database was loaded using the `files.upload()` and subsequently processed with `pandas`. During this process, invalid entries such as `[deleted]` or empty texts were removed. The final set of descriptions was stored in the list `textos`.

The following dictionary was also constructed:

$$te = \{\text{"texto_i"} : t_i\}$$

which associates each description with a unique identifier. This dictionary made it possible to retrieve the texts from their corresponding nodes within the graph.

VII-C. Construction of the Semantic Graph

An empty graph was created using NetworkX, and nodes were added for each element of the dictionary `te`. Subsequently, a **complete graph** was generated by adding an edge between every pair of distinct nodes:

$$E = \{(a, b) \mid a \neq b, a, b \in N(G)\}$$

This ensured that all descriptions were compared with one another.

VII-D. Vectorization Using Embeddings

Each description was transformed into a numerical vector using the Qwen2 7B Embedding model through the Unsloth library. To maintain consistency during inference, a template was used that integrates the text into an instruction:

$$\text{Instruct: } I \quad \text{Corpus: } t_i$$

A continuación, se generó la representación vectorial normalizada para cada nodo mediante:

$$\text{Vec}(t_i) = \frac{v_i}{\|v_i\|_2}$$

donde v_i es el embedding crudo producido por el modelo.

VII-E. Semantic Similarity Calculation

Once the vectors for each text had been generated, the semantic distance between each pair of graph nodes was calculated. For each edge (a, b) , the Euclidean distance between the embeddings was first obtained:

$$d(a, b) = \|\text{Vec}(a) - \text{Vec}(b)\|_2$$

Based on this distance, a weight value was defined and used solely for visualization purposes to represent how similar or different two texts are. This weight was calculated as follows:

$$w(a, b) = \frac{1}{0.5 + d(a, b)}$$

It is important to note that:

- A high value of $w(a, b)$ indicates semantically close texts.
- Un valor bajo de $w(a, b)$ indicates semantically very different texts.
- These values do not determine the position of the nodes in the visualization, but only the thickness of the edges.

The calculated weights were stored in the list **values**, which was subsequently used as the **width** parameter when drawing the graph, allowing the semantic strength between pairs of texts to be visually distinguished.

VII-F. Spatial Layout Using ForceAtlas2

To obtain a more comprehensible visual representation of the fully connected graph, the ForceAtlas2 algorithm was used. This method generates a spatial layout based exclusively on attractive and repulsive forces between nodes, without considering the previously calculated similarity values.

Therefore, it is essential to clarify that:

ForceAtlas2 does not use semantic similarity to position the nodes.

Its sole purpose is to provide a balanced visualization that avoids node overlap in dense graphs.

El resultado del algoritmo se almacenó en un diccionario:

```
pos = ForceAtlas2_layout(G)
```

which contains the two-dimensional coordinates for each graph node.

VII-G. Final Graph Visualization

Using the generated positions and calculated weights, the graph was visualized with **networkx**. The configuration used was:

- **node_size** = 10: small nodes to avoid visual clutter.

- **alpha** = 0.4: edge transparency.
- **width** = **values**: thickness proportional to semantic weight.
- **with_labels** = **False**: hide labels for greater clarity.

This visualization makes it possible to identify:

- Thick edges: similar texts.
- Thin edges: very different texts.

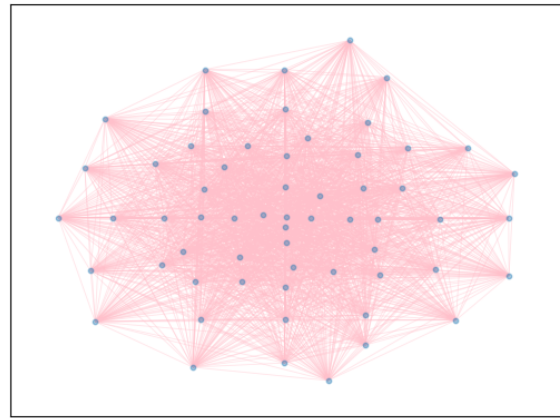


Figure 2: Final graph visualization with ForceAtlas2.

VII-H. Accessing Graph Information

The dictionary **te** made it possible to access the original contents associated with each node. This was particularly useful after identifying pairs of texts with extreme weights.

Since the texts **texto_11** y **texto_46** had the lowest semantic weight in the graph, their content was inspected to validate their conceptual difference using:

```
print(te["texto_11"])
print(te["texto_46"])
```

```
print(te["texto_11"])
... I might be a very unusual guy. My hobbies and interests
I'm just a honest, and lonely guy who distracts himself
```

Figure 3: Contenido del nodo **texto_11**.

```

print(te["texto_46"])
... I know it's getting real tough for you now but *60* hour work weeks AND a full
I failed out JC while working less than part time and volunteering (weekly). N
&nbsp;
&nbsp;
I had some small trouble on how to start this. You and most people start with
28 IT "support" guy that has dull and kind of secluded life interspersed with
Like you and many people who have commented (and internet citizens in general)
&nbsp;
Random but I miss hard liquor. Been drinking beer recently and, while the taste
I'm pretty involved in the alt-right movement, strengthening immigration laws,

```

Figure 4: Contenido del nodo `texto_46`.

Manual review confirmed that both texts differ substantially in topic, structure, and descriptive content, consistent with their low vector similarity.

VIII. RESULTS OBTAINED

The system successfully constructed a fully connected semantic graph from the processed texts. Each pair of nodes was connected by a weight calculated according to the distance between their embeddings.

The results showed a broad distribution of weights:

- High weights: close semantic relationships.
- Low weights: conceptually distant texts.

The analysis revealed that the pair consisting of:

(`texto_11`, `texto_46`)

corresponds to the lowest level of semantic similarity. Direct inspection of their contents confirmed this difference, validating the model and the implemented procedure.

The ForceAtlas2 visualization produced a readable graph diagram, although its layout is aesthetic rather than semantic. However, edge thickness did highlight relevant patterns, such as:

- regions with thick edges (similar texts),
- regions with thin edges (very different texts).

These results correctly reflect the internal semantic structure of the corpus used.

IX. CONCLUSIONS

The FINDER project demonstrated that combining semantic embeddings and graph theory makes it possible to effectively represent and analyze conceptual relationships between texts.

The vectors generated by the model captured the semantic essence of the corpus, and the use of a weighted graph made it possible to clearly quantify and visualize these relationships. The conceptual corrections applied during the review ensured an accurate interpretation of the results.

Identifying the pair with the lowest semantic similarity confirmed the system's ability to distinguish texts with very different topics. Likewise, the graph visualization made it possible to observe different patterns.

Overall, FINDER constitutes a valid and robust tool for studying semantic similarity, opening opportunities for thematic analysis, classification, or clustering in future projects.

X. REFERENCES

- Stryker Cole, Holdsworth Jim (s.f). What is NLP (natural language processing)?. IBM. <https://www.ibm.com/es-es/think/topics/natural-language-processing>
- Pablo Huet (12 de julio de 2024). Embeddings: What are they and how do they transform data into information? OpenWebinars. <https://openwebinars.net/blog/embeddings/>
- EBSCO (s.f). vector space. EBSCO. <https://www.ebsco.com/research-starters/physics/vector-space>
- Chugani Vinod (1 de octubre de 2024). Understanding Euclidean Distance: From Theory to Practice. <https://www.datacamp.com/es/tutorial/euclidean-distance>
- NLP Cloud (s.f). Semantic Similarity API. <https://nlpccloud.com/es/nlp-semantic-similarity-api.html>
- Attenborough Maria (2003). Graph Theory. ScienceDirect. <https://www.sciencedirect.com/topics/computer-science/weighted-graph>
- Zertuche Oriol (2024). The 8 Best Text Embedding Models in

2024.cody.<https://meetcody.ai/es/blog/los-8-mejores-modelos-de-incrustacion-de-texto-en-2024/>

- Holdsworth Jim (s.f). What is deep learning?. IBM. <https://www.ibm.com/es-es/think/topics/deep-learning>
- DataScientist (2022). Word2Vec: NLP Word Embedding. DataScientist. <https://datascientest.com/es/nlp-word-embedding-word2vec-es>
- Kaur Gurneet (2023). The Unexpected Love Affair: How AI Transforms Tinder's Dating Experience?. Analytics Vidhya. <https://www.analyticsvidhya.com/blog/2023/05/the-unexpected-love-affair-how-ai-transforms-tinders-dating-experience/>
- Geeks for Geeks (2025). GloVe Word Embedding in NLP. <https://www.geeksforgeeks.org/nlp/Glove-Word-Embedding-in-NLP/>
- Kashyap Kathrani (2020). All About Embeddings. Medium. <https://medium.com/@kashyapkathrani/all-about-embeddings-829c8ff0bf5b>